

Flight Radar

2D aircraft tracking — deck.gl · MobX · SharedWorker

Setup

Prerequisites

- A MapTiler API key (free tier works)

Install

```
git clone <repo-url>
cd flight-radar
npm install
```

Environment

Create a .env file in the project root:

```
VITE_REACT_KEY=your_maptiler_api_key_here
```

Get a free key at maptiler.com/cloud.

Run

```
npm run dev
```

Open <http://localhost:5173> in your browser.

What it does

- Renders 20 aircraft on a flat 2D map centred on the Middle East
- Colors each plane by country of origin — blue for Israel, red for Syria / Iran / Lebanon, orange for all others
- Clicking a plane opens an edit form on the right — rename the plane and hit Save
- A data panel at the bottom lists all tracked aircraft with name, country, and coordinates
- Selecting a plane in either the map or the panel keeps both views in sync
- Panning or zooming the map re-queries the worker so only planes inside the current viewport are rendered
- Selecting a plane in the panel flies the map to its position if it is outside the current viewport

Architecture

Three layers, each with a single responsibility.

SharedWorker — data layer

All plane data lives in the SharedWorker. It maintains a `planesById` Map and handles three messages: `INGEST_PLANES` to load data, `QUERY_VIEWPORT` to filter by bounding box, and `RENAME_PLANE` to update a name and broadcast the change to all connected tabs. The UI never mutates plane data directly.

MobX store — UI state layer

The store holds three values: `allPlanes` (full list, used by the panel), `visiblePlanes` (viewport subset, used by the map layer), and `selectedPlaneId`. `workerClient.ts` is the only bridge between the worker and the store.

deck.gl — rendering layer

The map uses `deck.gl` core directly with no `deck.gl-react` wrapper. A MobX reaction in `useRadarDeck.ts` watches `visiblePlanes` and `selectedPlaneId` and calls `deck.setProps()` on every change, keeping rendering entirely outside React's lifecycle.

Why visiblePlanes is not refreshed after rename

After a rename the worker broadcasts `ALL_PLANES`, which updates the panel and the edit form. It does not send a new `VISIBLE_PLANES` message. This is intentional — the `IconLayer` renders no name label. It uses only position, color, size, and heading. No visual on the map changes when a plane is renamed, so triggering a layer re-render would have no effect.

Selection

Selection never touches the worker. Clicking a plane calls `planesStore.selectPlane(id)` directly. The MobX reaction redraws the layer with the updated `selectedId`, growing the icon and raising its opacity. The panel syncs via `rowSelectionModel`.

When a plane is selected from the panel and its position lies outside the current viewport, the map animates to it using `map.flyTo()`. If the plane is already visible, the map does not move.

This is driven by a second MobX reaction in `useRadarDeck.ts` that watches `selectedPlane` and calls `map.flyTo()` only when needed.

Viewport query — complexity and trade-offs

The current implementation scans every plane in the worker on each viewport query:

```
for each plane:
  if plane is inside bbox → add to result
```

This results in $O(n)$ time per query, where n is the total number of planes. For the scope of this assignment and datasets in the hundreds to low thousands, this approach is simple, easy to follow, and sufficiently fast.

If the dataset grew significantly — tens of thousands of planes or more — scanning the full set on every pan and zoom could become a bottleneck. In that case, the worker could introduce a spatial index such as a grid partition, quadtree, or R-tree. A spatial index organizes planes by geographic region so that a viewport query only inspects the relevant areas instead of the entire dataset. Depending on the index structure, query complexity can be reduced to approximately:

- $O(c + k)$ for a grid-based index, where c is the number of grid cells intersecting the viewport
- $O(\log n + k)$ for tree-based indexes such as quadtrees or R-trees

where n is the total number of planes and k is the number of planes inside the viewport.

The current architecture supports this optimization without any UI changes — the worker interface remains the same, and only the internal filtering logic would be replaced.

Project structure

```
src/
  components/
    RadarMap/           map container + deck.gl hook
    PlaneEditor/        selected plane edit form
    PlanesPanel/        full aircraft data grid
  domain/
    plane.types.ts      Plane and GeoLocation types
    plane.color.ts      country → RGB mapping
    plane.normalize.ts  raw JSON validation
  layers/
    planesIconLayer.ts  deck.gl IconLayer factory
```

```
store/  
  planes.store.ts      MobX observable store  
services/  
  workerClient.ts      worker message bridge  
workers/  
  plane.worker.ts      SharedWorker — data owner  
data/  
  sample_planes_with_heading.json
```

Stack

- React TypeScript + Vite
- deck.gl rendering (core only, no deck.gl-react)
- MapLibre GL base map tiles via MapTiler
- MobX + mobx-react-lite — reactive state
- MUI — UI components and DataGrid
- SharedWorker — background data coordination